Urban Ride-Sharing Fleet Analytics

High-Velocity Telemetry Data Processing Using Apache Cassandra

Students:

Prathith Suresh Rao

S Gaja Lakshmi

S Nagashree

Saanvi S

Guiding Professor:

Leelavathi. B

Assistant Professor

Department of Computer Science

Project Overview

Urban Ride-Sharing Fleet Analytics

A Big Data Analytics project simulating and analyzing high-velocity telemetry streams from a live urban fleet environment using Apache Cassandra as the core data infrastructure.

Platform

Apache Cassandra NoSQL wide-column store

Domain

Urban ride-sharing fleet telemetry

Scale

50 vehicles · 500 rides · 8 city zones

Focus

Write throughput, analytics, concurrency

The Ride-Sharing Data Challenge

Mobility platforms like Uber and Ola generate **millions of continuous data points** every second — GPS coordinates, ride events, and dynamic pricing signals — at a scale traditional relational systems were never designed to handle.

Write Bottlenecks

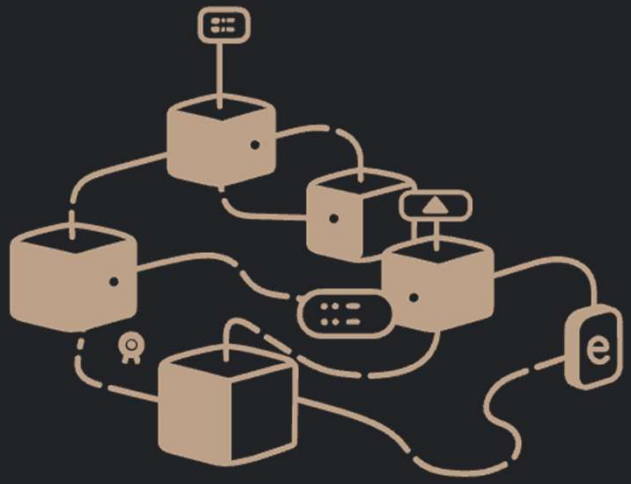
Relational databases choke under massive, concurrent write loads from live telemetry streams.

Expensive JOINs

Dynamic fleet matching requires cross-table queries — costly JOIN operations introduce unacceptable latency.

Concurrency Locks

Simultaneous driver status updates create database-level locks, causing race conditions and stale reads.



WHY CASSANDRA?

Built for Scale, By Design

Apache Cassandra is a **masterless, peer-to-peer NoSQL wide-column store** engineered to handle distributed, high-throughput workloads without a single point of failure.

High Availability

Peer-to-peer architecture with no master node — every node is equal, eliminating single points of failure.

Write Throughput

LSM-tree storage engine enables massive sequential write throughput ideal for IoT and telemetry workloads.

Time-Series Native

Composite primary keys enable natural, efficient modeling of time-stamped event data at fleet scale.

Project Scope and Dataset

A synthetic urban fleet environment was constructed to simulate real-world ride-sharing operations across 8 city zones in Bangalore, with full data portability demonstrated via CSV import/export pipelines.

8

City Zones

Geographically partitioned urban segments

50

Vehicles

Unique registered drivers in the fleet

500

Rides

Time-stamped completed ride records

Python

Generator

Synthetic dataset via scripted simulation

ARCHITECTURE · PART 1

NoSQL Features for Business Logic

Cassandra's data model was leveraged to solve core fleet analytics challenges natively at the database layer, eliminating the need for complex application-level logic.

Time-Series Modeling

Data partitioned by city zone and clustered by timestamp, enabling instant traffic burst analytics with O(1) partition lookups. Queries never scatter across nodes.

Auto-Expiring Surge Pricing

Surge pricing multipliers are stored with a Time-To-Live (TTL) of 30 minutes. Records self-delete after expiry — no backend cron jobs or scheduled cleanup tasks required.

ARCHITECTURE · PART 2

Concurrency and Dynamic Attributes

Handling simultaneous writes and schema-flexible vehicle attributes — two problems that cripple relational systems — are solved elegantly in Cassandra's data model.

Conflict-Free Counters

Cassandra's native CRDT Counter columns atomically track each driver's lifetime ride count. No read-modify-write cycles, no locks, and no race conditions — even under heavy concurrency.

Collection Types

List collections store dynamic vehicle amenities (WiFi, dashcams, child seats) directly in the row. Instant filtering is possible without any relational JOINs or auxiliary tables.

ANALYTICAL INSIGHTS · COMMUTER TRENDS

Business Intelligence via CQL

- ## High-Value Ride Zones

  Identified rides exceeding Rs. 400 fare threshold, pinpointing the most lucrative geographic corridors for surge pricing strategy.

- ## Commuter Exhaust Mapping

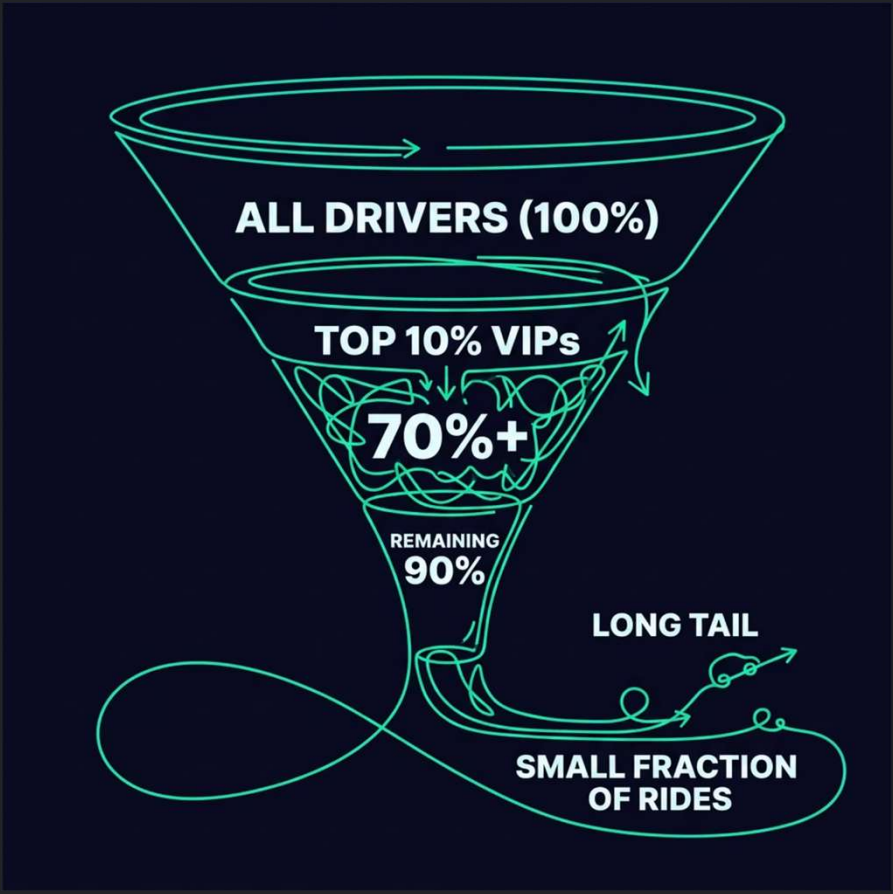
  Filtered extreme long-distance trips originating from Electronic City tech parks to model commuter fatigue and demand patterns.

- ## O(1) Partition Queries

  Executed traffic burst analytics on the Koramangala partition with constant-time latency — no full table scans, regardless of dataset size.

ANALYTICAL INSIGHTS · FLEET PERFORMANCE

Driver Distribution and Inventory Matching



Power-Law Driver Activity

Analysis revealed a classic **power-law distribution** in ride completions — a small cohort of high-frequency "VIP" drivers accounts for the vast majority of total fleet rides.

Corporate Fleet Segmentation

Using Cassandra's List collection filtering, corporate-ready vehicles (SUVs equipped with WiFi) were segmented from the fleet with **near-zero query latency** — no JOIN operations required. This validates the collection-type schema design for real-world inventory matching.

Conclusion

Apache Cassandra proves to be the **optimal architectural choice** for high-velocity, geographically distributed telemetry workloads. This project validates its suitability across every dimension of the ride-sharing analytics problem.

1

Zero Application Overhead

TTL and CRDT counters eliminate cron jobs and locking logic entirely from the application layer.

2

Native IoT & Telemetry Fit

LSM-tree writes and composite partition keys are purpose-built for time-series fleet data at scale.

3

Distributed by Default

Masterless peer-to-peer topology ensures fault tolerance and linear scalability across city zones.